

VibeScan

Documentação da Sprint 1

Diagramas C4 + Arquitetura Hexagonal + Estrutura do DAS

Tabata dos Santos Costa | Orientador: Reinaldo Galvão | Agosto de 2026

Sprint 1 — Levantamento Arquitetural e Estrutura Inicial

Esta sprint tem como objetivo o levantamento e a documentação dos componentes arquiteturais da solução VibeScan, com a criação dos diagramas C4 nos níveis 1 e 2, a documentação da Arquitetura Hexagonal com seus Ports e Adapters, e a estruturação inicial do Documento Arquitetural do Sistema (DAS).

Atividades da Sprint 1

- Diagrama C4 nível 1 — Contexto do Sistema
- Diagrama C4 nível 2 — Containers internos
- Documentação da Arquitetura Hexagonal (Ports & Adapters)

C4 Nível 1 — Contexto do Sistema

O diagrama de Contexto mostra o VibeScan no contexto organizacional, identificando os atores humanos que interagem com o sistema e os sistemas externos dos quais ele depende. Este é o nível mais alto de abstração do C4 Model.

[Inserir aqui a figura do Diagrama C4 Nível 1]

Figura X — Diagrama C4 Nível 1: Contexto do Sistema VibeScan

Descrição dos elementos

Elemento	Descrição e responsabilidade
Lucas Ferreira	Analista de Negócio Sênior — usuário gerador de código via ferramentas de vibe coding. Submete o código e o prompt original ao VibeScan e recebe o diagnóstico e o prompt melhorado.
Fernanda Rocha	Tech Lead Backend — revisora de código. Recebe o relatório diagnóstico estruturado para embasar a decisão de aprovação ou devolução.
VibeScan	Sistema central de análise — arquitetura de referência para governança e validação de software gerado por IA. Coordena o ciclo completo de análise, diagnóstico e geração de prompt melhorado.
Claude API (Anthropic)	Provedor de LLM externo. Recebe o prompt estruturado pelo VibeScan e retorna o diagnóstico em formato JSON com problemas categorizados, score e sugestões.
Ferramenta de IA	Sistema externo de vibe coding (Cursor, Copilot, Bolt.new). O usuário usa essa ferramenta para gerar o código e, após receber o prompt melhorado, para regenerar o código corrigido.
Repositório	Sistema de controle de versão (Git / Azure DevOps). Destino final do código aprovado após passar pelo processo de governança do VibeScan.

C4 Nível 2 — Containers

O diagrama de Containers detalha o interior do VibeScan, mostrando os processos executáveis e os mecanismos de armazenamento. Cada container é implantado separadamente e se comunica via interfaces bem definidas.

[Inserir aqui a figura do Diagrama C4 Nível 2]

Figura X — Diagrama C4 Nível 2: Containers do VibeScan

Descrição dos containers

Container	Tecnologia / Responsabilidade
-----------	-------------------------------

Blazor WebAssembly	Frontend web executado no browser. Interface para submissão de código e prompt, visualização do diagnóstico (score, problemas por dimensão, prompt melhorado) e histórico de análises.
API REST — ASP.NET Core	Backend da aplicação. Recebe requisições HTTP, coordena os Use Cases, expõe endpoints: POST /api/analise, GET /api/historico, GET /api/historico/{id}.
Domain + Application	Núcleo da arquitetura. Contém as entidades (Analise, ProblemaEncontrado), Value Objects (ScoreQualidade, CategoriaProblema), Use Cases (AnalisarCodigoUseCase, ObterHistoricoUseCase) e os Ports (interfaces).
Adapters	Implementações concretas dos Ports de saída: ClaudeAnalizadorAdapter (integração LLM), MemoryCacheRepositoryAdapter (persistência), PromptBuilderAdapter (montagem do prompt), AnaliseMapper (mapeamento DTO).
Cache em Memória	Mecanismo de persistência via IMemoryCache (.NET built-in). Armazena o histórico de análises da sessão com sliding expiration de 8 horas.
Claude API	Sistema externo. Recebe o prompt estruturado via HTTPS e retorna JSON com resumo, score, arquitetura recomendada, prompt melhorado e lista de problemas.

Arquitetura Hexagonal — Ports & Adapters

O VibeScan adota a Arquitetura Hexagonal (Ports & Adapters) como padrão central. Essa escolha garante que o núcleo do domínio — contendo as regras de negócio e os Use Cases — nunca dependa diretamente de infraestrutura. Toda dependência aponta para dentro do hexágono.

[Inserir aqui a figura da Arquitetura Hexagonal]

Figura X — Arquitetura Hexagonal: Ports & Adapters do VibeScan

Ports de Entrada (Inbound)

Port (Interface)	Implementado por
IAnalisarCodigoUseCase	AnalisarCodigoUseCase — orquestra o fluxo completo: cria o aggregate, chama o analisador, registra o resultado e persiste.
IObterHistoricoUseCase	ObterHistoricoUseCase — consulta e retorna o histórico de análises armazenadas no repositório.
IAnalisarDiretorioUseCase	AnalisarDiretorioUseCase — coordena a análise de múltiplos arquivos de um diretório, reutilizando IAnalisarCodigoUseCase por arquivo.

Ports de Saída (Outbound)

Port (Interface)	Adapter concreto / Responsabilidade
IAnalizadorPort	ClaudeAnalizadorAdapter — envia o prompt via HTTP à Claude API e parseia o JSON retornado via ClaudeResponseParser.
IAnaliseRepositoryPort	MemoryCacheRepositoryAdapter — persiste e consulta análises no IMemoryCache com controle de concorrência via lock.
IPromptBuilderPort	PromptBuilderAdapter — monta o prompt estruturado enviado ao LLM, incluindo contexto de linguagem e prompt original do usuário.
IArquivoScannerPort	FileSystemScannerAdapter — lê arquivos de código do sistema de arquivos local, filtra por extensão e ignora pastas desnecessárias (bin, obj, node_modules).

Princípios SOLID aplicados

Princípio	Onde é aplicado no VibeScan
S — Single Responsibility	Cada classe tem uma responsabilidade: PromptBuilderAdapter monta prompt, ClaudeAnalizadorAdapter chama a API, ClaudeResponseParser parseia a resposta.
O — Open/Closed	Novo provedor de LLM (ex: OpenAI): criar OpenAiAnalizadorAdapter e registrar no DI — sem alterar nenhuma linha do domínio.
L — Liskov Substitution	ClaudeAnalizadorAdapter substitui IAnalizadorPort em qualquer contexto sem quebrar o comportamento esperado pelo Use Case.
I — Interface Segregation	IAnalizadorPort, IAnaliseRepositoryPort e IPromptBuilderPort são interfaces separadas e específicas — nenhuma implementação carrega métodos que não usa.
D — Dependency Inversion	AnalisarCodigoUseCase depende de IAnalizadorPort (abstração), nunca de ClaudeAnalizadorAdapter (implementação concreta).

Design Patterns Aplicados

Pattern	Aplicação no VibeScan
Adapter	ClaudeAnalizadorAdapter e MemoryCacheRepositoryAdapter implementam interfaces do domínio, adaptando infraestrutura externa para o contrato esperado.

Factory Method	Analise.Criar() — método factory estático que valida invariantes e garante que um Aggregate Root nunca seja criado em estado inválido.
Repository	IAnaliseRepositoryPort abstrai a persistência. Hoje: IMemoryCache. Futuro: SQL Server ou Redis — sem tocar no domínio.
Strategy	Os analisadores especializados (segurança, arquitetura, qualidade, testabilidade) são estratégias intercambiáveis na coordenação do ciclo de análise.
DTO	Separação entre entidades de domínio (Analise) e contratos da API (AnalisarCodigoResponse) — nenhum controller acessa propriedades internas do Aggregate.
Value Object	ScoreQualidade (garante 0–10 com classificação e cor) e CategoriaProblema (instâncias estáticas imutáveis) são Value Objects sem identidade própria.

Estrutura do Documento Arquitetural do Sistema (DAS)

O DAS será construído incrementalmente ao longo das 3 sprints. Abaixo está a estrutura prevista do documento final, com indicação de qual sprint produz cada seção.

Seção do DAS	Sprint / Status
1. Introdução e Contexto	Sprint 1 — Desafio e Análise de Contexto
2. Objetivo Arquitetural	Sprint 1 — Objetivo SMART aprovado
3. Diagrama C4 Nível 1 — Contexto	Sprint 1 — Esta sprint
4. Diagrama C4 Nível 2 — Containers	Sprint 1 — Esta sprint
5. Arquitetura Hexagonal e Ports	Sprint 1 — Esta sprint
6. Ciclo de Análise e Melhoria	Sprint 2 — Próxima sprint
7. Motor de Regras e Decisão	Sprint 2 — Próxima sprint
8. Integração com o Provedor de LLM	Sprint 2 — Próxima sprint
9. Evidências de Execução	Sprint 2 — Prints e resultados reais
10. Validação Experimental (scores)	Sprint 3 — Comparação 1ª vs 2ª análise
11. Validação das Hipóteses H1–H6	Sprint 3 — Resultados observados
12. Considerações Finais	Sprint 3 — Lições aprendidas